

Energy-efficient Analog Integrated Circuits

PLL – Exercise 2

Giorgio Cristiano

Prof. Taekwang Jang

1. Introduction

In this exercise session we are going to explore the design procedure for a charge pump PLL (CPPLL), depicted in Figure 1. We will start from the design of the in-band contributors, namely phase detector and charge pump. Then, we will move to the VCO, and finally we will put everything together on MATLAB to better analyze the closed loop behaviour. The PLL will multiply a reference frequency of 25MHz by 80, obtaining an output frequency of 2GHz. Here we will focus on the VCO.

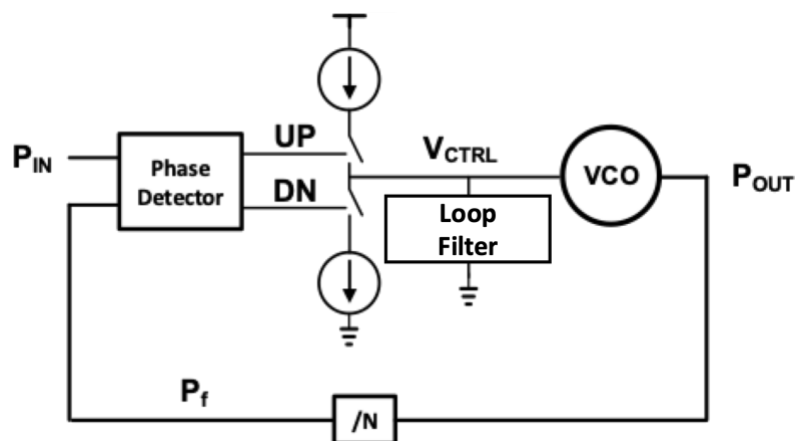


Figure 1: Block diagram of CPPLL

The Voltage-Controlled-Oscillator is the component that generates the output of the PLL. This will generate an output frequency dependent on the control voltage applied at the input. In a PLL, this control voltage is set by the feedback loop in order to have a constant (if type I PLL), or zero (if type II PLL) phase error between output and reference signals.

One way of designing a VCO is to put an odd number of inverters in a ring as shown in Fig.2, so that each node will continuously oscillate between LO and HI at each iteration. This is the so-called ring oscillator. The main advantage of this structure is its limited size, and its easiness of integration with other CMOS circuits. However, its output noise is limited by the MOS operation, and cannot be beyond a physical limit of around -165dBc/Hz.

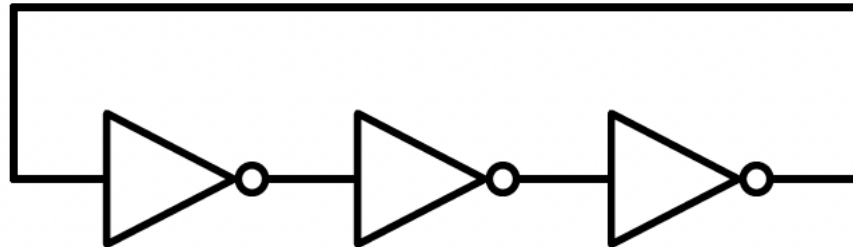


Fig.2: Ring oscillator

To achieve better noise performances, we can exploit the resonance of LC circuits, where each transition is helped by this resonance behaviour, achieving better noise performances. In this case, we talk about LC VCOs. The main drawbacks of this structure are found in the larger size, and a bit more care needed when integrating it in the rest of the circuit to not degrade excessively the performance of the inductor.

In this lab session we will design together a simple ring VCO, while the design of a LC VCO is left as an exercise.

2. Design of Ring VCO

Let's start by creating a new library called VCO, in which we will create a new schematic called *Inverter*. This schematic will be the unit delay cell used in our ring VCO. In this case we will use a simple inverter. Its schematic is shown in Fig.3. Create its symbol.

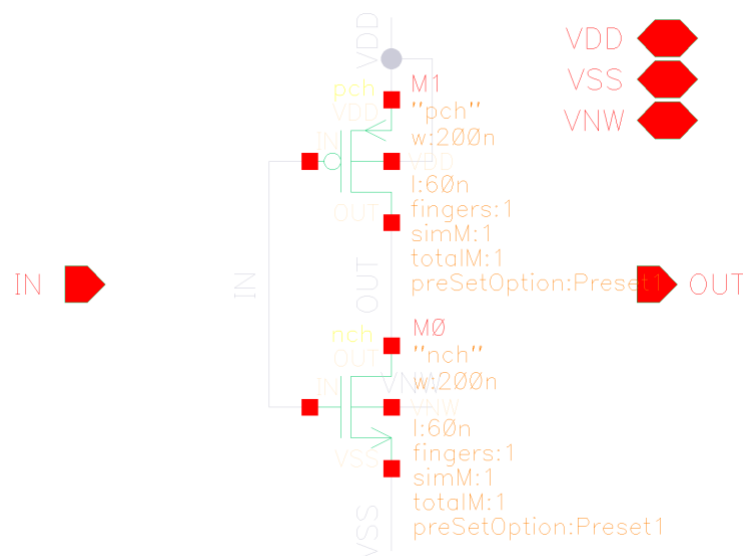


Fig 3: Schematic of Ring Inverter

Once we have the basic delay cell, we can design the VCO. Create a new schematic called Ring, and instantiate the components as shown in Fig. 4.

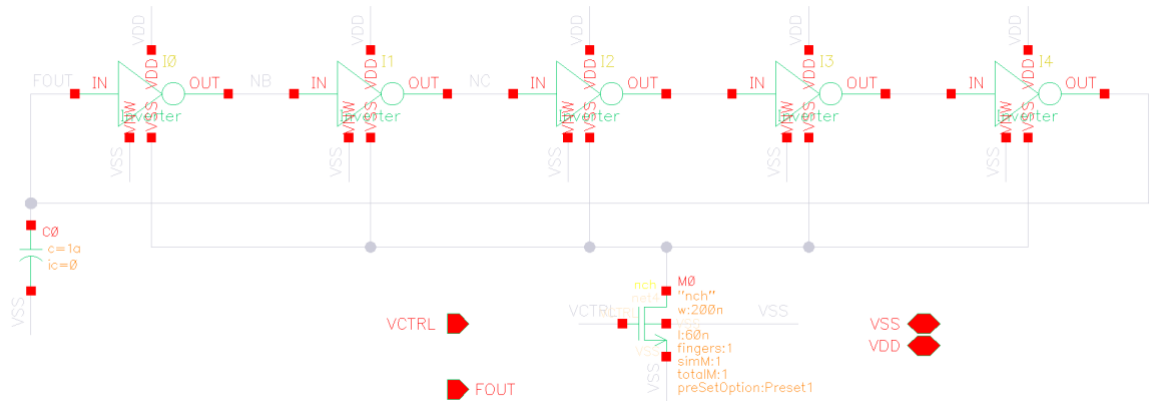


Fig.4: Schematic of Ring VCO

Note the capacitor attached to node *FOUT*. This very small capacitor (1aF) is a quick trick to force an initial condition of a node to the simulator. This will help us in the starting of the VCO. Note how this will never work in a real application and will have to be replaced with real devices.

Q2.1: What is the purpose of M0 in Fig.4? How does it work? Which swing do you expect at the output?

To restore the full swing, and buffer the VCO we will also add an AC coupled inverter. Create a new cellview called RingBuffer, and instantiate the components shown in Fig.5.

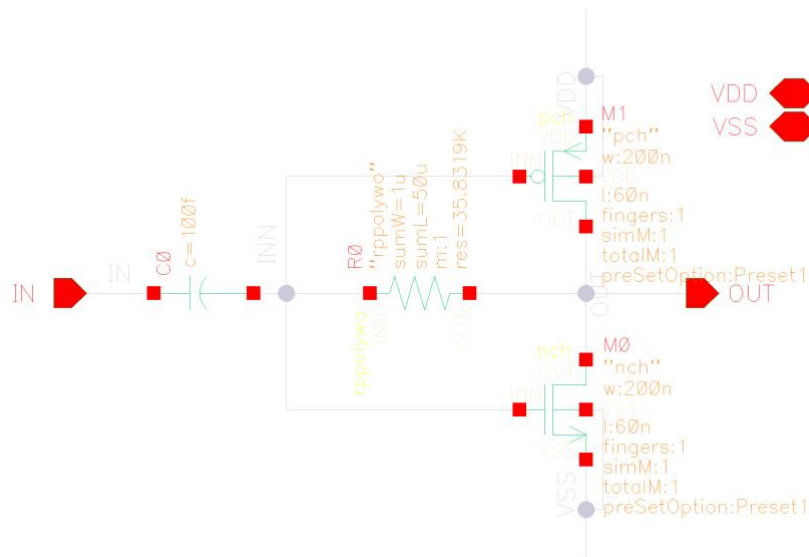


Fig.5: Ring AC Buffer

Q2.2: How do you think I have chosen the R and C values for this buffer?

Let's now test this VCO. As usual, create a symbol for the Ring cellview. Then, create a new cellview called zTEST_Ring, and in the schematic instantiate the VCO just created. Tie VSS at 0V, VDD and VCTRL at values controllable through the testbench. Add the AC buffer at the Ring Output, and load it with a 10fF capacitor. The testbench is shown in Fig. 6.

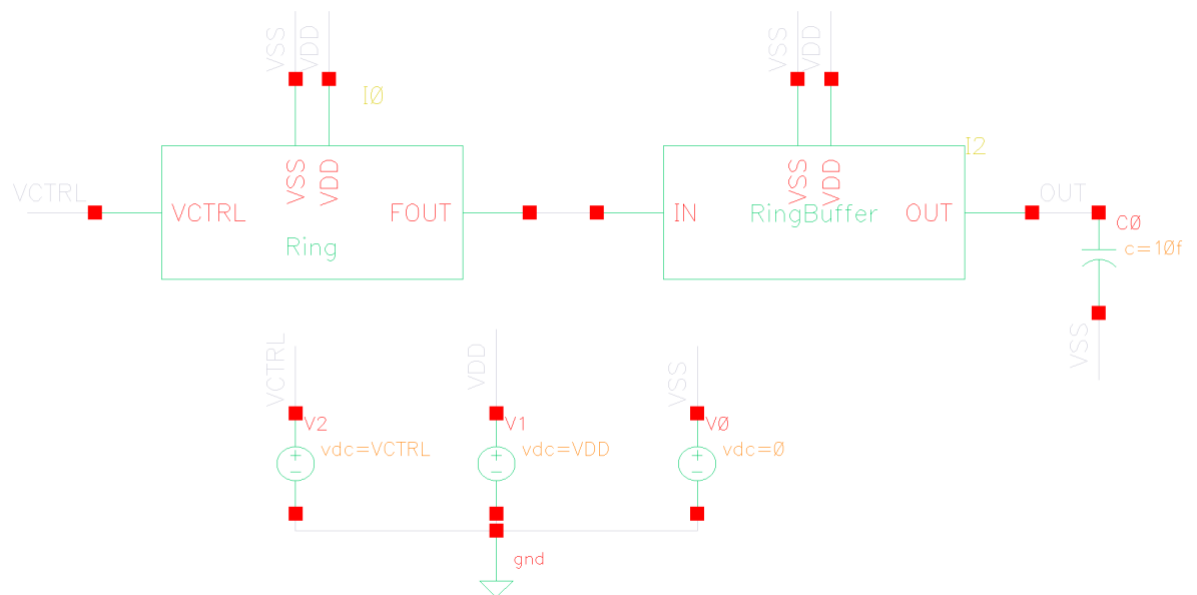


Fig 6: Ring VCO testbench

We can now move to the testing part. From the schematic view you can create a new ADE Assembler view from *Launch > ADE Assembler > Create New View*. Here, create a new test. We are interested in checking the beat frequency, power, and noise of the oscillator in steady-state. Since the steady-state is periodic we will have to simulate it with *pss* and *pnoise* analyses. These can be set similarly to how it was done for the chopping amplifier, with some minor differences. First of all, here we are dealing with an oscillator, so we cannot know the target frequency before-hand. Hence, in the *pss* setup we will indicate a rough frequency approximation, which could be gotten through a trans simulation, for instance. Then we will select the *Oscillator* option, selecting whatever node as positive node, and allowing the simulator to automatically calculate the initial conditions. The full *pss* setup is shown in Fig.7.1.

The *pnoise* setup will calculate the noise between 1kHz, and $f_{beat}/2$, as usual. However, we are here interested at the phase part of the noise, so we will consider PM modulation at the bottom of the *pnoise* setup. This means that we will have to run the sweep as relative. As we are interested in the noise around the main carrier, we will select the 1st relative harmonic. Again, the number of sidebands should be chosen with trial and error, making sure we are considering all the HF contributions. In this case we will keep it to 100. For simplicity we can keep the number of harmonics in the *pss* setup equal to the max sidebands in *pnoise*. The full *pnoise* setup is shown in Fig.7.2.

Periodic Steady State Analysis

Engine ☒ Shooting ☐ Harmonic Balance

#	Name	Expr	Value	Signal	SrcId

Large ☒ ☐

☒ Beat Frequency ☐ Beat Period

26 ☐

Output harmonics

Number of harmonics ☒ 100

Accuracy Defaults (errpreset)

☒ conservative ☐ moderate ☐ liberal

Transient-Aided Options

Run transient? ☒ Yes ☐ No ☐ Decide automatically

Detect Steady State ☒ Stop Time (tstab) 10n

Save Initial Transient Results (saveinit) ☐ no ☒ yes

Dynamic Parameter ☐

Oscillator ☒

Oscillator node+ /OUT

Oscillator node-

☒ Calculate initial conditions (ic) automatically

☐ Enable tuning mode analysis

Sweep ☐

New Initial Value For Each Point (restart) ☐ no ☐ yes

Loadpull ☐

Enabled ☒

Periodic Noise Analysis

PSS Beat Frequency (Hz) 26

Multiple noise ☐

Sweeptype ☒ relative ☐ Relative Harmonic 1

Output Frequency Sweep Range (Hz)

Start-Stop ☒ Start 1k Stop 26/2

Sweep Type

Automatic ☒

Add Specific Points ☐

Sidebands

Method ☒ default ☐ fullspectrum

Maximum sideband ☒ 100

When using shooting engine, default value is 7.

Output

voltage ☒ Positive Output Node /OUT

Negative Output Node

Noise Type ☒ timeaverage

Timeaverage: single-sided spectrum and harmonic-referred (modulated) noise analysis

Contribution Type:

☐ USB ☐ AM ☒ PM ☐ AM&PM ☐ ALL(AM,PM,USB,LSB)

For fmjitter, PM noise must be enabled

Noise Separation ☐

Separate noise into source and gain

Enabled ☒

Fig.7.1: pss setup

Fig.7.2: pnoise setup

We have now set the analyses. We can then go ahead and set the design variables. Fix VDD at 1.2V, and VCTRL at HVDD (0.6V). Then we can run the simulations. Once it is done we can, as usual, use the direct plot feature to plot the signals and expressions of interest. From the pss analysis plot the nodes *FOUT*, *I0/NB*, *I0/NC*, *OUT* versus time. Do the same with the current entering the VDD node of the Ring Oscillator. Check that they match with the plot in Fig.8. The beat frequency can be extracted from the pss as well, selecting the function *Harmonic Frequency* and selecting the first harmonic.

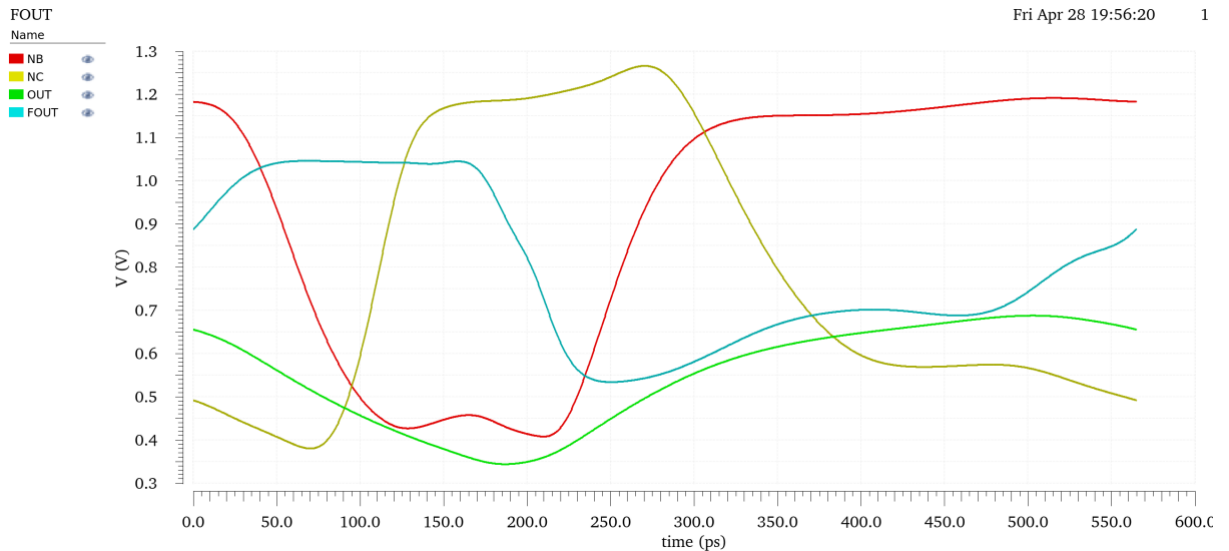


Fig.8: Ring VCO steady-state

To plot the power, we will average the current, and multiply it by VDD. To do so name the current expression as I, then the power will be given as $abs(average(I)*VAR("VDD"))$.

The next step is to plot the noise. Use again the Direct Plot, and from the pnoise analysis select the output noise, in V^2/Hz , measured in SSB. Plot the noise graph and make sure it. Matches the one in Fig.9.

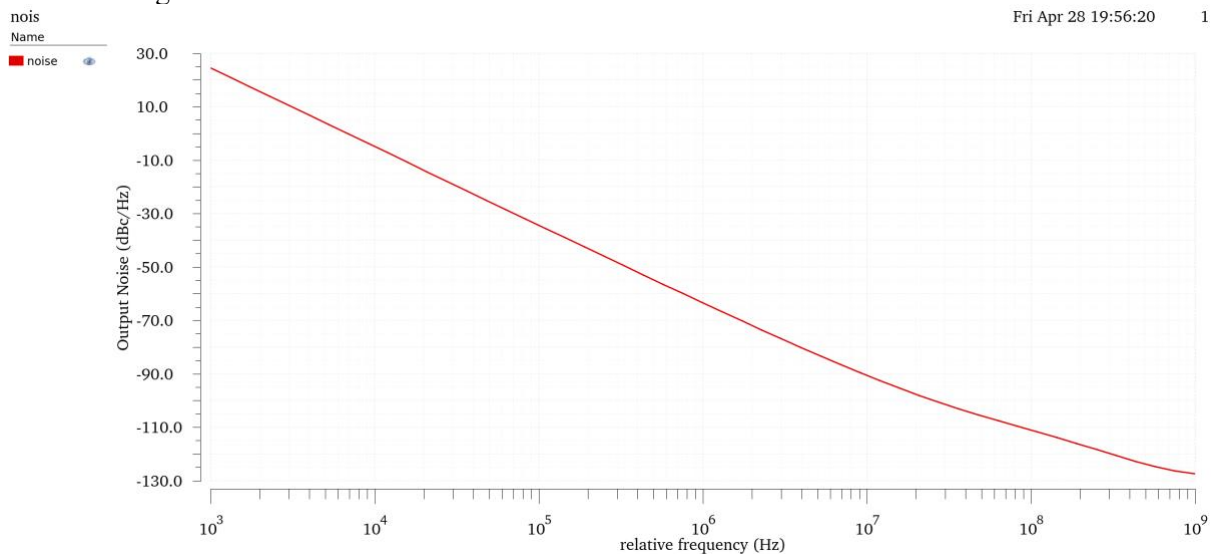


Fig.9: Noise of Ring VCO

Now we should have all the data needed to calculate the FoM of this oscillator.

Q2.3: What is the FoM for a VCO? How is it calculated? What does this value represent?

Calculate the FoM at 100k and 1M from beat frequency putting the FoM formula in the Cadence calculator.

Make sure that all the values you have obtained match the ones shown in Fig.10.

VCO:zTEST_Ring:1	P	20.16u
VCO:zTEST_Ring:1	fbeat	1.77G
VCO:zTEST_Ring:1	noise_100k	-34.17
VCO:zTEST_Ring:1	noise_1M	-63.27
VCO:zTEST_Ring:1	FoM_100k	-136.1
VCO:zTEST_Ring:1	FoM_1M	-145.2

Fig.10: Results of VCO analysis

3. Sizing of Ring VCO

As you probably have noticed, the transistors have been purposefully left in min size for the point above. In fact, the results shown by the simulation are very far off from what we will need for a PLL, and the waveforms do not make much sense. You will now size everything correctly.

A3.1: Keeping the same VCO structure as before, size all transistors so that the VCO:

- 1. Consumes 2mW ($\pm 0.1\text{mW}$)**
- 2. Has a beat frequency of 2GHz($\pm 100\text{MHz}$), for a VCTRL of 0.75V**
- 3. Has a FoM of -162dBc/Hz or better at 1MHz offset from carrier.**
- 4. Has a FoM of -158dBc/Hz or better at 100kHz offset from carrier.**
- 5. The OUT correctly swings between VSS and VDD**

Describe and justify your choices.

Hint: To check the noise contributors at a specific frequency (or frequency interval), you can use Plot>Noise Summary, as we have done for the Charge Pump.

Note: In this design we are keeping the tail control transistor as a single unit for simplicity. This leads to a very large VCO gain K_{VCO} . In a real PLL design, in which the K_{VCO} needs to be correctly tuned, this transistor will be split in multiple devices in parallel. Then, most of them will be connected to a fixed potential, and only a minority will be connected to the tuning node VCTRL. This will strongly reduce the K_{VCO} .

A3.2: What are the major noise contributors? How do you address each?

A3.3: Plot the noise graph. Describe each region, which kind of noise dominates in each, and what info you can gather from them.

4. Design of LC VCO

Now we will apply this design procedure to the classB LC VCO shown in Fig.11.

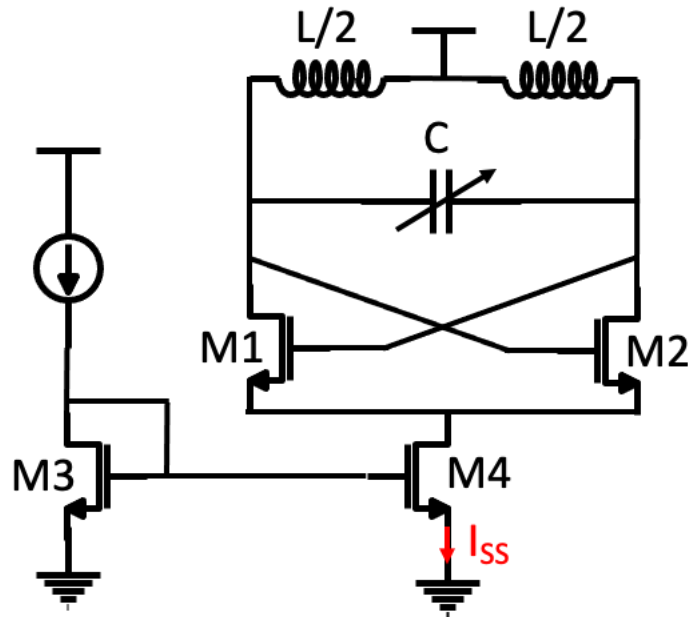


Fig.11: ClassB VCO

A4.1: Describe the functionality of each component in Fig.11

For the ClassB design we start from the choice of inductor. Generally, this requires an in depth study of all the inductors available in the technology. For this exercise we will use the *indq* instance available in *analogLib*. This cell allows to describe an inductor by means of its inductance L , and its quality factor Q . Fix Q at 15.

The inductance can then be chosen by knowing the wanted voltage swing of the outputs. The single-ended swing is described by $V_{swing} = \frac{2}{\pi} I_{SS} R_P$, where I_{SS} is the tail current of the VCO, and R_P the parallel resistance of the tank. At this point we will make an approximation, and say that R_P is the parallel resistance of the inductor.

A4.2: Calculate the R_P needed to have 0.6V single-ended swing. Then, using the Q of the inductor calculate the needed inductance L at an oscillation frequency of 2GHz and $I_{SS}=2mA$.

A4.3: What is the drawback if the swing is too large? What if it is too small?

After the sizing of the inductors, you can move to the sizing of the XCP. Theoretically, the minimum boundary for these transistors is the one that allows oscillation (360deg phase shift at resonance frequency), which happens when $(g_m R_P) \geq 1$. Intuitively this can be also understood by realizing that the negative g_m cell must re-inject the energy dissipated by the parasitic resistance of the tank.

However, this calculation is true only for small-signal excitations, thus when the VCO swing is small. For large swings, the XCP will be fully switched, and for part of the period its g_m will fall below $1/R_P$. Hence, we design the XCP so that g_m is larger than $1/R_P$. What we are looking for is that the transistors are big enough to steer all the current on one branch for each half cycle. However we must be careful, as a XCP too big will also lead to issues. The sizing of a class B VCO is very delicate. On one hand, large XCP and large swing will cause the XCP to enter triode for part of the operation, leading to an increase of the noise, on the other hand, making the XCP too small will lower the V_{ds} of M4. This is also not desirable, as it will increase the

sensitivity to the supply. During the design phase one must also make sure that M4 is in saturation.

A4.4: As seen above, the transistors of the XCP need a large W/L. But what about the length value L? How would you fix the length of those transistors? Why?

Finally, we can set the capacitance of the tank. This, of course need to be chosen so that it resonates at the target frequency of 2GHz. However, we must be careful, and realize that there are other parasitic capacitances that load the tank, namely the inductor capacitance (not present in this case as we are using an ideal inductor), and the capacitances of the XCP.

Moreover, for a real application, we need tunability, because PVT variation will cause the LC to resonate at frequencies different from target, and we need a way to re-center the VCO. This is generally done with digitally switching capacitor banks, as shown in Fig.12.

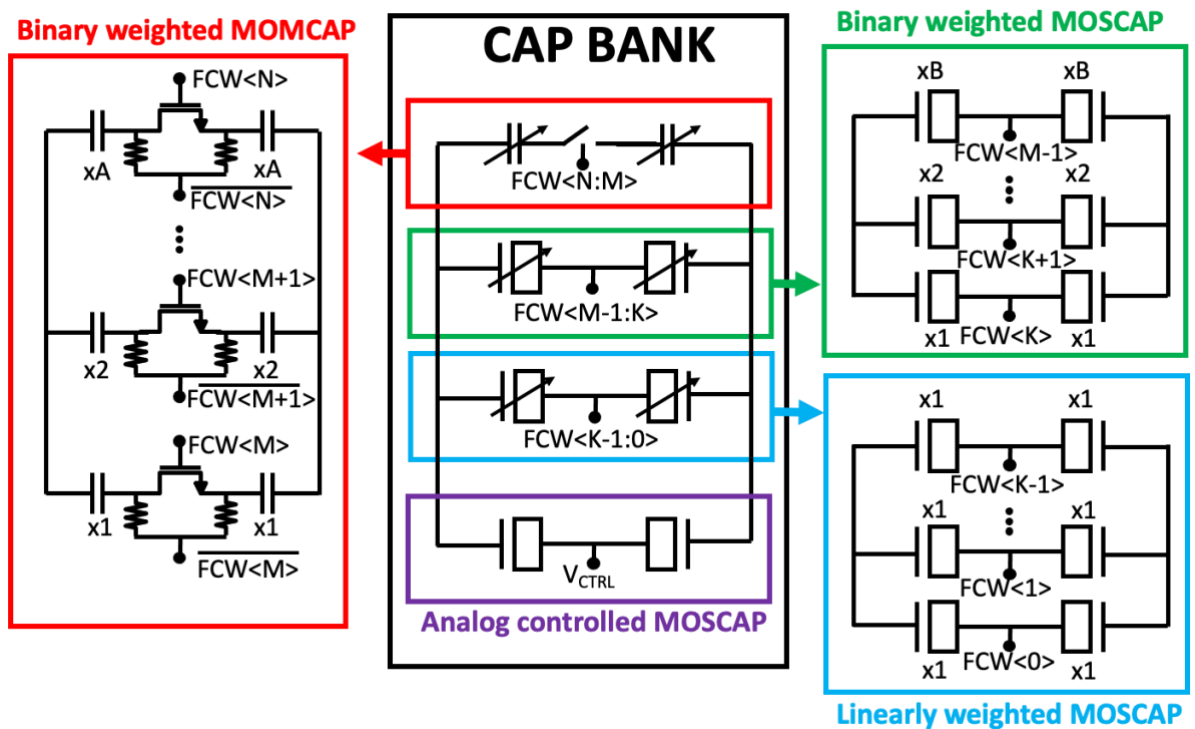


Fig.12: Capacitor banks for LC VCO

Note how we divide the capacitors in multiple banks. Starting from the bottom, we have a MOS varactor connected to a V_{CTRL} voltage. That is sized to have the wanted VCO gain K_{VCO} for the PLL loop.

Just above there is are two banks of digitally controlled MOS varactors. These are controlled by the LSBs of the frequency control word (FCW). The least significant bank is linearly controlled to minimize issues coming from variation of MOSCaps during fabrication, while the most significant bank is binary weighted.

Finally, we find a bank of MOM caps that are controlled by the MSBs of the FCW. Note the peculiar switch implementation. This is done to ensure the correct turn on and off of the transistor. In our application, the gate is switched between 0-1.2V, while the S and D have the DC point fixed to VSS when the transistor is on, and to 0.6V when the transistor is off.

We want to avoid as much as possible holes in the possible tuneable frequencies. To do so we must put extra care at the interface between banks (MOM-MOSbin, MOSbin-MOSlin, MOSlin-MOSanalog). As a rule of thumb, it is good to make the banks “overlap” by setting the capacitance of the LSB of the larger bank equal to the capacitance of the MSB of the smaller bank.

There is a clear trade-off between tuning range, and achievable FoM. In this design we will keep the tuning range quite relaxed, meaning that only a small section of the total tank capacitance will need to be switchable. The rest can be implemented as a fixed capacitance to center the oscillating frequency to the target.

Note that in this technology there is no provided MOM cell. Generally, we would layout one, extract it, and use that. Instead, in this exercise, for simplicity, we will just use the MIM cap cell *mimcap_3t* in the library *tsmcN65*, with the bulk connection to ground.

A4.5: Design the capacitor bank for the VCO.

- 1. Calculate the total capacitance C_{res} needed to resonate at 2GHz, using the L value found previously.**
- 2. We want to have a tuning range of $\pm 5\%$ centred at the target frequency. Calculate C_{max} and C_{min} as the capacitances that would resonate at $f_{target}-5\%$ and $f_{target}+5\%$ respectively.**
- 3. How would you size the switches that control the MIM caps? What happens if the switch is too small? What if it is too big?**

Finally, the last spec that we have not yet discussed in terms of functionality is the gain of the VCO K_{VCO} . This parameter is extremely important, as it reflects directly on the BW of the PLL loop, and thus must be controlled accurately. The analog controlled moscaps need to be sized so that $K_{VCO}=df/dV_{ctrl}$ meets the target spec. In this design we will fix $K_{VCO}=5\text{MHz/V}$.

A4.6: Design a ClassB VCO so that:

- 1. It resonates at 2GHz, with a $I_{SS}=2\text{mA}$, and a single sided swing $>0.5\text{V}$**
- 2. Has a FoM of -187dBc/Hz or better @ $f_{off}=100\text{kHz}$**
- 3. Tuneable at least in the range $[1.9\text{GHz}-2.1\text{GHz}]$ with overlapping capacitive banks. Show your whole range, and that you do not have tuning holes with a sweep.**
- 4. Has a gain $K_{VCO}=5\text{MHz/V}$.**
- 5. Show that your tuning range has no holes and you have enough bank overlap when passing from one bank to the other.**

Describe your design process, and justify the decisions you have taken.

Note1: In the points above we have approximated R_p to the parallel resistance of the inductor alone. Once the capacitor bank is added to the design, it also adds some contribution. The calculations you have done above provide you a good starting point, but you might need to reiterate the design values a little bit to meet all the specs.

Note2: when considering the tuning range, changing the frequency will change also parameters such as the signal swing. Make sure the constraints are met @2GHz. Then, the purpose of the tuning is to have flexibility when taking into account PVT variation, but the VCO will always go at locking frequency. Hence it is ok if the signal swing is different far away from the centre frequency.

Note3: The design of the capacitive bank is actually tricky. For simplicity, let's keep the MOS bank fully binary weighted. Start designing the tunable MOM part with ideal switches. You can then replace those with real components, meeting the specs for extra points.

Note4: You can use the following VerilogA script to convert a decimal to binary, and use it to easily change the frequency control word (FCW) in your testbench. To use it make a new cellview, and in a VerilogA view adapt the script below to your needs (e.g. Change NBITS to the value you need). Check and save it, and create a symbol. Then, instantiate the symbol in the testbench. In its properties, at *CDF Parameter of view* select *verilog*, and under the voice *inval* put a name of a variable that you will control from maestro.

```
`include "constants.vams"
`include "disciplines.vams"
(* ignore_hidden_state *) module dec2bin_ClassD(Vdd_bCTRL, Vdd_CTRL, Vss,
CTRL, bCTRL);

input Vdd_bCTRL, Vdd_CTRL, Vss;
electrical Vdd_bCTRL, Vdd_CTRL, Vss;
output [0:NBITS-1] CTRL;
electrical [0:NBITS-1] CTRL;
output [0:NBITS-1] bCTRL;
electrical [0:NBITS-1] bCTRL;

parameter integer inval = 0;

real tr = 100f;
integer res, next;
real CTRL_sig[0:NBITS-1], bCTRL_sig[0:NBITS-1];
genvar i, j;

analog begin
    @(initial_step) begin
        res = 0;
        next = inval;

        for(i=0; i<NBITS; i=i+1)begin
            res = next % 2;
            next = (next-res) / 2;
            CTRL_sig[i] = res*V(Vdd_CTRL);
            bCTRL_sig[i] = (1-res)*V(Vdd_bCTRL);
        end
    end

    for(i=0; i<NBITS; i=i+1) begin
        V(bCTRL[i],Vss) <+ transition(bCTRL_sig[i], 0, tr, tr);
    end

    for(j=0; j<NBITS; j=j+1) begin
        V(CTRL[j],Vss) <+ transition(CTRL_sig[j], 0, tr, tr);
    end

end

endmodule
```